DEPARTAMENTO DE FÍSICA

Universidad Técnica Federico
Santa María
Campus Santiago San Joaquín
FIS190

Proyecto FIS190: Ecualizador de 3 bandas automatizado mediante RaspBerryPi y FFT

Autores. Paul Berdichewsky[1]
Benjamín Arancibia[1]

[1] *Departamento de Física, Universidad Técnica Federico Santa María, San Joaquín, Chile*

1. Introducción

El procesamiento de señales de audio constituye una de las áreas más importantes de la electrónica moderna, debido a su amplia aplicación en sistemas de comunicaciones, instrumentación, multimedia y electrónica de consumo. Una señal de audio está compuesta por múltiples componentes de frecuencia distribuidas a lo largo del espectro audible, por lo que en numerosas aplicaciones resulta necesario modificar selectivamente determinadas bandas de frecuencia con el objetivo de mejorar la calidad del sonido, reducir interferencias o adaptar la respuesta del sistema a un propósito específico.

Los dispositivos encargados de realizar esta tarea reciben el nombre de *ecualizadores*, los cuales emplean filtros electrónicos para modificar la amplitud de distintas regiones del espectro de frecuencias. En los ecualizadores tradicionales, la selección de estos filtros es realizada manualmente por el usuario. Sin embargo, el desarrollo de sistemas embebidos de bajo costo ha permitido incorporar técnicas de procesamiento digital de señales para automatizar este proceso.

El presente proyecto propone el desarrollo de un sistema capaz de seleccionar automáticamente el filtro más adecuado según el contenido espectral de una señal de audio. Para ello se utilizó una Raspberry Pi Pico como unidad de control, encargada de adquirir una copia de la señal mediante su convertidor analógico-digital (ADC), analizarla utilizando un algoritmo basado en la Transformada Rápida de Fourier (FFT) y controlar un conjunto de relés que seleccionan el filtro correspondiente.

Es importante señalar que el diseño de la etapa de filtrado no fue desarrollado desde cero en este proyecto. Los filtros activos utilizados fueron implementados siguiendo el trabajo presentado por Fanco, Quintero y Ardila en *Ecualizador de 3 bandas basado en filtros activos de segundo orden* 1. En dicho trabajo se desarrolla un ecualizador analógico de tres bandas basado en filtros activos de segundo orden utilizando la configuración Sallen–Key. En el presente proyecto se tomó dicho diseño como referencia para implementar la etapa analógica de filtrado, incorporando como aporte original una etapa de análisis digital y control automático mediante Raspberry Pi Pico, permitiendo seleccionar dinámicamente el filtro adecuado sin intervención del usuario.

De esta manera, la arquitectura propuesta combina procesamiento analógico y procesamiento digital. Mientras los filtros activos realizan el procesamiento de la señal de audio, la Raspberry Pi Pico actúa únicamente como sistema de análisis y control, automatizando la selección del filtro mediante un sistema de relés. Esta integración permitió aplicar conocimientos de electrónica analógica, procesamiento digital de señales y sistemas embebidos en una única solución orientada al procesamiento inteligente de señales de audio.

2. Objetivos

2.1. Objetivo General

Diseñar e implementar un sistema automático de ecualización de audio capaz de analizar el contenido espectral de una señal mediante una Raspberry Pi Pico y seleccionar automáticamente uno

de tres filtros activos de segundo orden utilizando un sistema de relés.

2.2. Objetivos Específicos

- Diseñar e implementar filtros activos pasa bajos, pasa banda y pasa altos utilizando la topología Sallen–Key para cubrir las regiones de graves, medios y agudos del espectro audible.
- Implementar una etapa de adquisición de la señal utilizando el convertidor analógico-digital (ADC) integrado en la Raspberry Pi Pico.
- Desarrollar un algoritmo basado en la Transformada Rápida de Fourier (FFT) capaz de estimar la distribución espectral de la señal de audio en tiempo real.
- Controlar automáticamente la selección de los filtros mediante módulos de relé accionados desde las salidas digitales de la Raspberry Pi Pico.
- Integrar las etapas analógicas y digitales en una única arquitectura de procesamiento híbrido, evaluando el comportamiento del sistema durante su funcionamiento.

3. Descripción General del Sistema

El sistema desarrollado fue concebido como una arquitectura híbrida que combina procesamiento digital y procesamiento analógico para realizar una selección automática de filtros de audio. El objetivo principal consiste en analizar continuamente una señal de entrada e identificar cuál es la banda de frecuencia predominante, permitiendo seleccionar automáticamente el filtro más apropiado para su procesamiento.

La señal de audio es obtenida desde un dispositivo externo, como un teléfono móvil o un computador, mediante un conector auxiliar de 3.5 mm. Posteriormente, la señal es dividida en dos caminos independientes.

El primer camino corresponde al procesamiento digital. Una copia de la señal es acondicionada para ser adquirida por el convertidor analógico-digital (ADC) integrado en la Raspberry Pi Pico. Debido a que el ADC únicamente admite tensiones comprendidas entre 0 y 3.3 V, la señal debe ser previamente desplazada mediante una etapa de polarización (bias), permitiendo que el microcontrolador pueda adquirir correctamente las muestras de audio.

El segundo camino corresponde al procesamiento analógico. La señal original es enviada hacia un conjunto de módulos de relé que actúan como interruptores electrónicos. Dependiendo de la decisión tomada por la Raspberry Pi Pico, únicamente uno de estos relés permanece activado, dirigiendo la señal hacia el filtro correspondiente.

Una vez adquiridas las muestras, el microcontrolador ejecuta un algoritmo basado en la Transformada Rápida de Fourier (FFT), el cual permite obtener la distribución espectral de la señal. A partir de esta información se calcula la energía presente en las bandas de graves, medios y agudos. Finalmente, el sistema activa únicamente el relé asociado a la banda de mayor energía, permitiendo que la señal sea procesada por el filtro correspondiente.

Esta arquitectura permite mantener el procesamiento principal de la señal completamente en el dominio analógico, mientras que el procesamiento digital se utiliza únicamente para realizar el análisis espectral y controlar la selección automática de filtros.

4. Diseño de los Filtros

La etapa de procesamiento analógico fue implementada mediante filtros activos de segundo orden utilizando la configuración Sallen–Key. Esta topología fue seleccionada debido a su simplicidad, estabilidad y facilidad de implementación utilizando amplificadores operacionales, además de ofrecer una pendiente de atenuación de aproximadamente 40 dB por década, superior a la obtenida mediante filtros de primer orden.

El diseño de los filtros fue realizado tomando como referencia la metodología propuesta por Franco, Quintero y Ardila [**franco2010**], quienes desarrollan un ecualizador activo de tres bandas basado en filtros Sallen–Key para aplicaciones de audio.

En el presente proyecto se implementaron tres filtros independientes, correspondientes a las bandas de graves, medios y agudos. La selección entre estos filtros es realizada automáticamente por la Raspberry Pi Pico mediante un sistema de relés, evitando que el usuario deba seleccionar manualmente el procesamiento de la señal.

4.1. Filtro Pasa Bajos

El filtro pasa bajos fue diseñado con una frecuencia de corte aproximada de 250 Hz, permitiendo el paso de las componentes de baja frecuencia correspondientes principalmente a sonidos graves. Para su implementación se utilizó una red Sallen–Key de segundo orden basada en un amplificador operacional TL084.

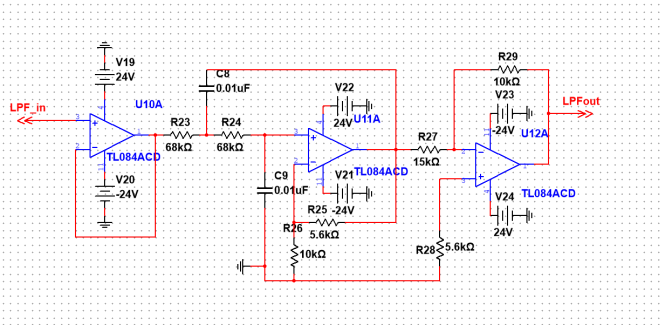


Figura 1: Filtro pasa bajos implementado en Multisim.

4.2. Filtro Pasa Banda

El filtro pasa banda fue diseñado para permitir únicamente el paso de las componentes de frecuencia comprendidas entre aproximadamente 500 Hz y 2 kHz. Este rango corresponde principalmente a las frecuencias medias presentes en señales de voz e instrumentos musicales, mientras que atenúa tanto las componentes de baja frecuencia como las de alta frecuencia.

El filtro fue implementado utilizando dos etapas activas de segundo orden en configuración Sallen–Key conectadas en cascada: un filtro pasa altos con frecuencia de corte inferior de 500 Hz y un filtro pasa bajos con frecuencia de corte superior de 2 kHz. De esta forma se obtiene una banda de paso centrada en las frecuencias medias, proporcionando una mejor selectividad que la obtenida mediante filtros de primer orden.

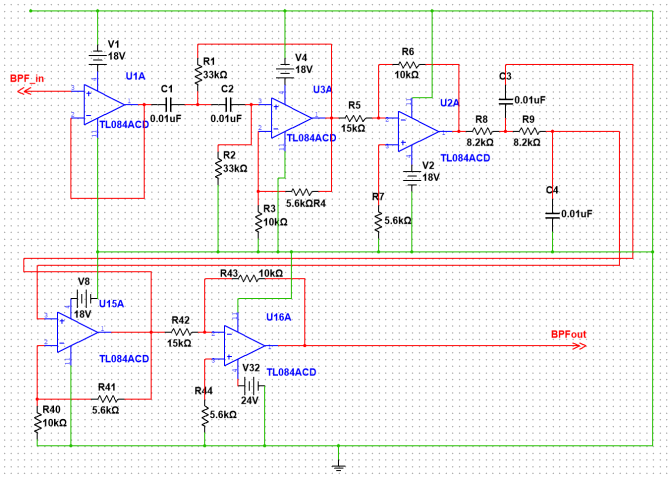


Figura 2: Filtro pasa banda implementado en Multisim.

4.3. Filtro Pasa Altos

El filtro pasa altos fue diseñado con una frecuencia de corte aproximada de 4 kHz, permitiendo el paso de las componentes de alta frecuencia presentes en la señal de audio y atenuando aquellas ubicadas por debajo de dicha frecuencia. Este filtro resulta útil para aislar sonidos agudos y reducir la influencia de componentes de baja frecuencia.

Al igual que los demás filtros implementados en el proyecto, se utilizó una configuración Sallen–Key de segundo orden basada en un amplificador operacional TL084. La elección de esta topología proporciona una mayor pendiente de atenuación respecto a filtros de primer orden, mejorando la separación

entre las distintas bandas de frecuencia.

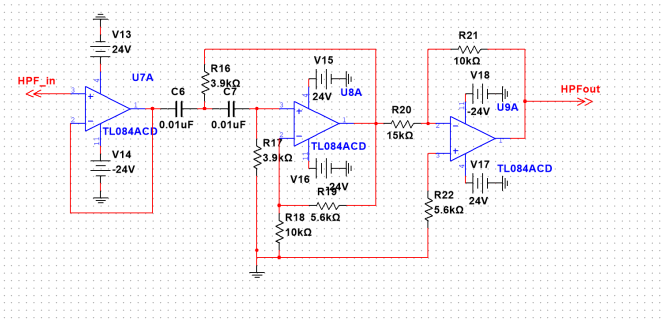


Figura 3: Filtro pasa altos implementado en Multisim.

5. Funcionamiento del Sistema

El funcionamiento del sistema puede dividirse en cuatro etapas principales: acondicionamiento de la señal de entrada, análisis espectral y selección del filtro, procesamiento analógico mediante filtros activos y acondicionamiento de la señal de salida. La Figura 4 muestra el esquemático general implementado.

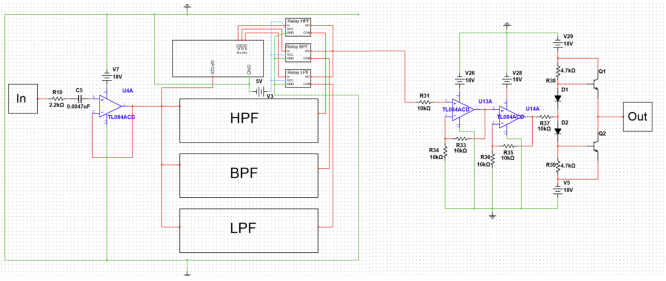


Figura 4: Esquemático general del sistema desarrollado.

5.1. Etapa de entrada

La señal de audio ingresa al sistema mediante un conector auxiliar de 3.5 mm proveniente de un dispositivo externo, como un teléfono móvil o un computador. Antes de ser procesada, la señal atraviesa una etapa de acoplamiento formada por un capacitor y un amplificador operacional configurado como seguidor de voltaje (buffer).

El capacitor de entrada elimina cualquier componente continua presente en la señal, evitando que ésta afecte el funcionamiento de las etapas posteriores. Posteriormente, el buffer implementado mediante un amplificador operacional TL084 adapta las impedancias entre la fuente de audio y el resto del circuito, evitando pérdidas de amplitud y aislando eléctricamente el dispositivo de entrada de las etapas de procesamiento.

A la salida del buffer, la señal es dividida en dos caminos. El primero corresponde al análisis digital realizado por la Raspberry Pi Pico, mientras que el segundo alimenta simultáneamente las entradas de los tres filtros analógicos.

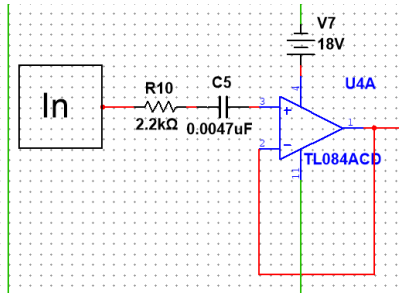


Figura 5: Esquemático de la etapa de entrada.

5.2. Etapa de análisis y selección

La Raspberry Pi Pico adquiere una copia de la señal utilizando su convertidor analógico-digital (ADC). A partir de las muestras obtenidas, el microcontrolador ejecuta un algoritmo basado en la Transformada Rápida de Fourier (FFT), permitiendo estimar la energía contenida en distintas regiones del espectro de frecuencias.

El programa compara la energía correspondiente a las bandas de graves, medios y agudos, determinando cuál presenta la mayor contribución espectral. En función de este resultado, la Raspberry Pi Pico activa únicamente uno de los tres módulos de relé conectados a sus salidas digitales, seleccionando automáticamente el filtro que procesará la señal de audio.

Esta estrategia permite automatizar completamente la selección del filtro sin intervenir directamente sobre la señal analógica.

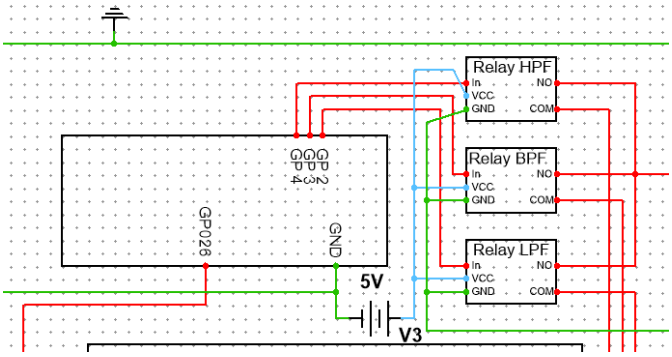


Figura 6: Esquemático de la etapa de análisis y selección.

5.3. Etapa de filtrado

El procesamiento principal de la señal se realiza mediante tres filtros activos de segundo orden implementados utilizando la topología Sallen–Key.

Cada filtro fue diseñado para actuar sobre una banda específica del espectro audible:

- Filtro pasa bajos (LPF), encargado de procesar las componentes graves de la señal.
- Filtro pasa banda (BPF), destinado al procesamiento de las frecuencias medias.
- Filtro pasa altos (HPF), utilizado para las componentes de alta frecuencia.

Aunque los tres filtros permanecen conectados al sistema, únicamente uno de ellos recibe la señal de audio en cada instante gracias a la conmutación realizada por los módulos de relé. De esta manera se evita que las respuestas de los distintos filtros se superpongan y se garantiza que el procesamiento corresponda únicamente a la banda seleccionada por el algoritmo.

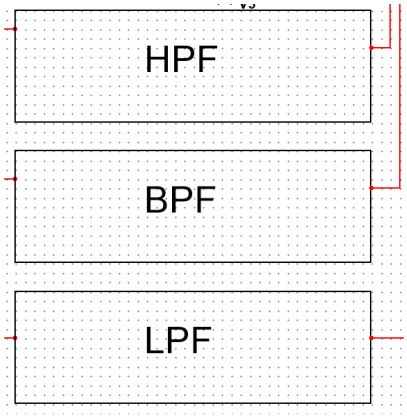


Figura 7: Esquemático de la etapa de filtrado.

5.4. Etapa de salida

Una vez filtrada, la señal es enviada a una etapa de acondicionamiento de salida implementada mediante dos amplificadores operacionales TL084 configurados como seguidores de voltaje y una etapa complementaria de potencia basada en transistores bipolares.

Los amplificadores operacionales realizan nuevamente una adaptación de impedancias, entregando una señal estable a la etapa de potencia y evitando que la carga conectada modifique la respuesta del filtro seleccionado.

Finalmente, la etapa push-pull implementada con los transistores Q1 y Q2 proporciona la corriente necesaria para alimentar la carga conectada a la salida del sistema, permitiendo reproducir la señal filtrada mediante un parlante activo o cualquier dispositivo de audio conectado al conector de salida.

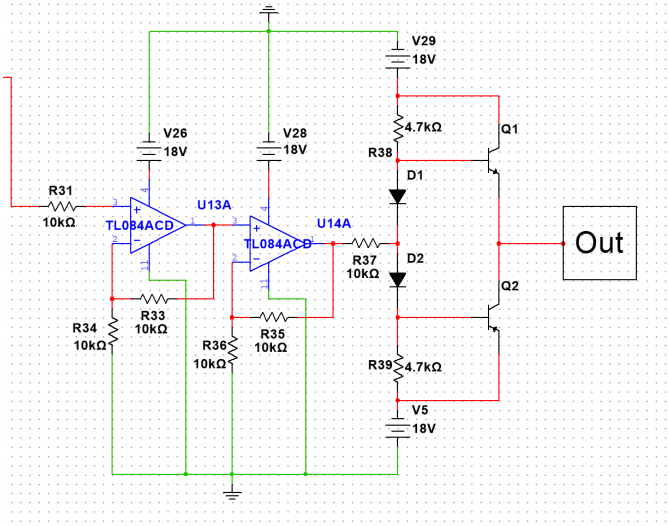


Figura 8: Esquemático de la etapa de salida.

6. Montaje Experimental

La implementación física del sistema se realizó sobre placas de pruebas (*protoboards*), permitiendo integrar las distintas etapas analógicas y digitales desarrolladas durante el proyecto. En el montaje se implementaron los filtros activos de segundo orden, la Raspberry Pi Pico encargada del análisis de la señal y los módulos de relé utilizados para la selección automática de los filtros.

La Figura 9 muestra una vista general del montaje experimental, donde es posible identificar la distribución de los distintos bloques que conforman el sistema.

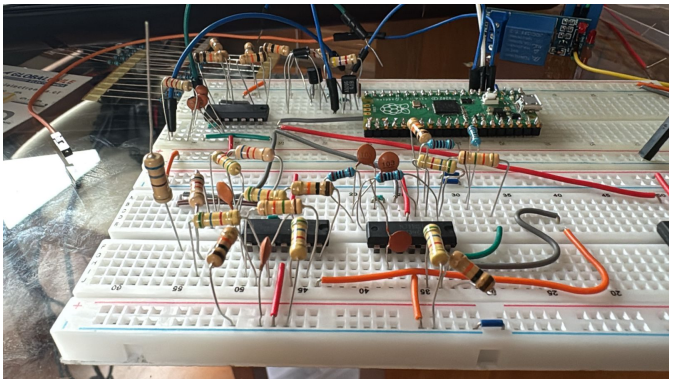


Figura 9: Vista general del montaje experimental del sistema.

La Figura 10 presenta un acercamiento a la implementación de la etapa de procesamiento, donde se observan los filtros activos construidos mediante amplificadores operacionales TL084, la Raspberry Pi Pico y los módulos de relé empleados para la selección automática del filtro.

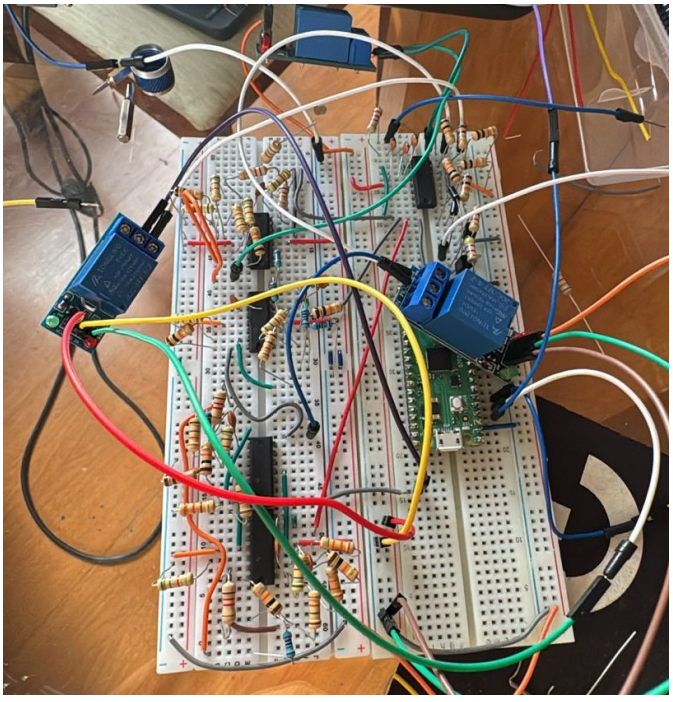


Figura 10: Detalle del montaje implementado sobre placa de pruebas.

7. Fundamento Matemático

El sistema utiliza dos herramientas principales: filtros activos analógicos para modificar la señal de audio y análisis espectral digital para decidir qué filtro debe ser seleccionado.

7.1. Frecuencia de corte de los filtros

Para un filtro de primer orden, la frecuencia de corte está determinada por la relación entre la resistencia y el condensador:

$$f_c = \frac{1}{2\pi RC} \tag{1}$$

donde f_c corresponde a la frecuencia de corte, R a la resistencia y C a la capacitancia. En filtros activos de segundo orden tipo Sallen–Key, esta relación se mantiene como base de diseño cuando se utilizan componentes equivalentes, aunque la respuesta final depende también de la ganancia del amplificador operacional y del factor de calidad Q.

La frecuencia angular de corte se expresa como:

$$\omega_c = 2\pi f_c \tag{2}$$

Para un filtro pasa bajos de segundo orden, la función de transferencia ideal puede escribirse como:

$$H_{LPF}(s) = \frac{\omega_c^2}{s^2 + \frac{\omega_c}{Q}s + \omega_c^2} \tag{3}$$

En el caso de un filtro pasa altos de segundo orden:

$$H_{HPF}(s) = \frac{s^2}{s^2 + \frac{\omega_c}{Q}s + \omega_c^2} \tag{4}$$

Para el filtro pasa banda, se considera una frecuencia de corte inferior f_L y una frecuencia de corte superior f_H. Su ancho de banda se define como:

$$BW = f_H - f_L \tag{5}$$

y su frecuencia central aproximada como:

$$f_0 = \sqrt{f_L f_H} \tag{6}$$

En este proyecto se consideraron tres regiones principales del espectro audible:

- Graves: $f < 250$ Hz.
- Medios: 500 Hz $< f <$ 2 kHz.
- Agudos: $f > 4$ kHz.

7.2. Transformada Discreta de Fourier

Para que la Raspberry Pi Pico pueda determinar la banda predominante de la señal, es necesario transformar la señal desde el dominio temporal hacia el dominio de la frecuencia. Para ello se utiliza la Transformada Discreta de Fourier (DFT), definida como:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \tag{7}$$

donde $x[n]$ representa las muestras de la señal en el tiempo, N es el número total de muestras adquiridas y $X[k]$ corresponde a la componente espectral asociada al índice k.

La frecuencia asociada a cada componente espectral está dada por:

$$f_k = \frac{k f_s}{N} \tag{8}$$

donde f_s corresponde a la frecuencia de muestreo del sistema.

La energía asociada a una componente frecuencial puede estimarse mediante:

$$E[k] = |X[k]|^2 \tag{9}$$

Luego, para cada banda de frecuencia, se puede calcular una energía total sumando las componentes espectrales pertenecientes a dicho rango:

$$E_{\text{banda}} = \sum_{k=k_1}^{k_2} |X[k]|^2 \tag{10}$$

Finalmente, el algoritmo compara las energías de las tres bandas:

$$E_{\max} = \text{máx}(E_{\text{graves}}, E_{\text{medios}}, E_{\text{agudos}}) \tag{11}$$

La banda asociada a $E_{\max}$ determina cuál relé debe ser activado y, por lo tanto, qué filtro procesará la señal de audio.

7.3. Criterio de selección

El criterio de decisión implementado puede expresarse como:

$$\text{Filtro} = \begin{cases} \text{Pasa bajos}, & \text{si } E_{\text{graves}} > E_{\text{medios}}, E_{\text{agudos}} \\ \text{Pasa banda}, & \text{si } E_{\text{medios}} > E_{\text{graves}}, E_{\text{agudos}} \\ \text{Pasa altos}, & \text{si } E_{\text{agudos}} > E_{\text{graves}}, E_{\text{medios}} \end{cases} \tag{12}$$

De esta manera, la selección del filtro queda determinada por el contenido espectral predominante de la señal de entrada.

8. Algoritmo de Control

La Raspberry Pi Pico fue utilizada como unidad de control del sistema, siendo la encargada de analizar la señal de audio y seleccionar automáticamente el filtro analógico correspondiente.

Al iniciar el programa se configuran los pines GPIO asociados a los módulos de relé y se inicializa el convertidor analógico-digital (ADC), encargado de adquirir las muestras de la señal de entrada. Posteriormente, el sistema ejecuta un ciclo continuo de procesamiento.

En cada iteración se obtiene un bloque de muestras de la señal de audio, el cual es procesado mediante un algoritmo basado en la Transformada Rápida de Fourier (FFT). El análisis espectral permite estimar la energía presente en las bandas de graves, medios y agudos.

Una vez obtenidas las energías de cada banda, el algoritmo identifica la componente predominante y desactiva todos los relés antes de activar únicamente el correspondiente al filtro seleccionado. Esta estrategia evita que dos filtros permanezcan conectados simultáneamente y garantiza que la señal sea procesada únicamente por una de las tres rutas disponibles.

El proceso se ejecuta continuamente mientras el sistema permanece encendido, permitiendo adaptar la selección del filtro a los cambios presentes en el contenido espectral de la señal de entrada.

```
Inicio

Configurar ADC
Configurar GPIO de los relés

While: el sistema esté activo:

    Leer muestras del ADC

    Calcular FFT

    Obtener energía:
        Graves
        Medios
        Agudos

    Comparar energías

    Desactivar todos los relés

    Activar el relé correspondiente

Fin While
```

9. Resultados y Discusión

Una vez implementadas las distintas etapas del sistema, se realizaron pruebas individuales con el objetivo de verificar el funcionamiento del hardware y del software antes de proceder a la integración completa.

En primer lugar, se comprobó el funcionamiento de los filtros activos de segundo orden implementa-

dos mediante la configuración Sallen–Key, verificando que cada uno respondiera al rango de frecuencias para el cual fue diseñado. Paralelamente, se desarrolló el algoritmo de control en la Raspberry Pi Pico, encargado de adquirir las muestras de la señal y seleccionar automáticamente el filtro correspondiente mediante el accionamiento de los módulos de relé.

Durante la etapa de integración se presentaron diversas dificultades asociadas a la interacción entre la etapa digital y la etapa analógica. Una de las principales limitaciones fue el acondicionamiento de la señal de audio para el convertidor analógico-digital (ADC) de la Raspberry Pi Pico, ya que fue necesario adaptar una señal alterna para trabajar dentro del rango de tensión permitido por el microcontrolador.

Asimismo, el análisis espectral presentó un comportamiento diferente al esperado cuando se trabajó con señales reales de audio. Factores como el ruido eléctrico, las variaciones en la amplitud de la señal y las limitaciones propias del sistema de adquisición dificultaron la identificación estable de la banda predominante, afectando el accionamiento automático de los relés.

A pesar de estas dificultades, el desarrollo permitió validar el diseño conceptual del sistema y comprender la importancia del acondicionamiento de la señal y de la correcta integración entre el procesamiento digital y el procesamiento analógico. Además, se verificó el funcionamiento individual de los módulos de relé, de las etapas de filtrado y del algoritmo de control, identificando los principales aspectos a mejorar para futuras implementaciones.

Los resultados obtenidos permitieron identificar las principales limitaciones asociadas a la integración entre el procesamiento analógico y digital. Aunque no fue posible alcanzar un funcionamiento completamente estable durante las pruebas finales, el desarrollo permitió validar la arquitectura propuesta y comprender los desafíos involucrados en la adquisición y clasificación automática de señales de audio. La experiencia obtenida constituye una base para futuras mejoras del sistema, particularmente en el acondicionamiento de la señal de entrada y en la optimización del algoritmo de análisis espectral.

10. Conclusiones

El presente proyecto permitió desarrollar una arquitectura híbrida para el procesamiento de señales de audio, integrando una etapa de análisis digital implementada en una Raspberry Pi Pico con una etapa de procesamiento analógico basada en filtros activos de segundo orden. A lo largo del desarrollo se aplicaron conocimientos relacionados con electrónica analógica, procesamiento digital de señales, sistemas embebidos y control mediante dispositivos electrónicos.

Se implementó un sistema capaz de analizar una señal de audio y seleccionar automáticamente uno de tres filtros mediante un conjunto de módulos de relé. Para ello, se diseñó un algoritmo encargado de adquirir la señal a través del convertidor analógico-digital de la Raspberry Pi Pico, estimar el contenido espectral y determinar la banda de frecuencia predominante. Asimismo, se implementó la etapa de filtrado utilizando filtros activos en configuración Sallen–Key, integrándolos dentro de una única arquitectura de procesamiento.

Durante la etapa de integración se presentaron dificultades relacionadas principalmente con el acondicionamiento de la señal para el convertidor analógico-digital, así como con la estabilidad del análisis espectral frente a señales de audio reales. Estas limitaciones impidieron obtener un funcionamiento completamente estable del sistema durante las pruebas finales. No obstante, el desarrollo permitió identificar los principales desafíos asociados a la integración entre procesamiento digital y analógico, así como comprender la importancia de un adecuado acondicionamiento de la señal y de la correcta sincronización entre las distintas etapas del sistema.

Como trabajo futuro, se propone mejorar la etapa de adquisición mediante un circuito de acondicionamiento más robusto o una interfaz de audio dedicada, optimizar el algoritmo de análisis espectral para aumentar su estabilidad y reemplazar la conmutación mediante relés por interruptores analógicos, reduciendo los tiempos de respuesta y las perturbaciones introducidas durante el cambio entre filtros.

En conclusión, si bien no fue posible alcanzar el funcionamiento completamente automático esperado dentro del tiempo disponible para el desarrollo del proyecto, se logró implementar y validar el diseño conceptual del sistema, integrando de manera satisfactoria sus principales etapas y estableciendo una base sólida para futuras mejoras y desarrollos.

11. Referencias

1. *FANCO, R. A., QUINTERO, E. A., ARDILA URUEÑA, W. (2010). ECUALIZADOR DE 3 BANDAS BASADO EN FILTROS ACTIVOS DE SEGUNDO ORDEN. Scientia Et Technica, XVI(45), 124-129.*

12. Anexo

Código de Python

```
from machine import ADC, Pin
import utime
import math

# Entrada ADC
```

```
adc = ADC(26)   # GPIO26 = ADC0

# Relés asociados a cada filtro
rele_graves = Pin(2, Pin.OUT)
rele_medios = Pin(3, Pin.OUT)
rele_agudos = Pin(4, Pin.OUT)

# Parámetros de muestreo
FS = 8000       # Frecuencia de muestreo aproximada
N = 256         # Número de muestras por bloque

# Frecuencias representativas de cada banda
FREQ_GRAVE = 250
FREQ_MEDIO = 1000
FREQ_AGUDO = 3000

ultimo_filtro = "medio"

# Cambiar a True si el módulo de relé es activo en LOW
RELE_ACTIVO_EN_LOW = False


def rele_on(rele):
    rele.value(0 if RELE_ACTIVO_EN_LOW else 1)


def rele_off(rele):
    rele.value(1 if RELE_ACTIVO_EN_LOW else 0)


def seleccionar_filtro(nombre):
    global ultimo_filtro

    # Apagar todos los relés antes de activar uno
    rele_off(rele_graves)
    rele_off(rele_medios)
    rele_off(rele_agudos)

    utime.sleep_ms(20)

    if nombre == "grave":
        rele_on(rele_graves)
    elif nombre == "medio":
        rele_on(rele_medios)
```

```
    elif nombre == "agudo":
        rele_on(rele_agudos)

    ultimo_filtro = nombre


def leer_bloque():
    samples = []
    periodo_us = int(1000000 / FS)

    for _ in range(N):
        valor = adc.read_u16()
        samples.append(valor)
        utime.sleep_us(periodo_us)

    # Eliminar componente DC
    promedio = sum(samples) / len(samples)
    samples = [x - promedio for x in samples]

    return samples


def goertzel(samples, freq, fs):
    k = int(0.5 + ((N * freq) / fs))
    omega = (2.0 * math.pi * k) / N
    coeff = 2.0 * math.cos(omega)

    q0 = 0
    q1 = 0
    q2 = 0

    for sample in samples:
        q0 = coeff * q1 - q2 + sample
        q2 = q1
        q1 = q0

    power = q1 * q1 + q2 * q2 - q1 * q2 * coeff
    return power


def decidir_filtro(grave, medio, agudo):
    if grave > medio and grave > agudo:
        return "grave"
    elif medio > grave and medio > agudo:
```

```
        return "medio"
    else:
        return "agudo"


seleccionar_filtro("medio")

while True:
    muestras = leer_bloque()

    energia_grave = goertzel(muestras, FREQ_GRAVE, FS)
    energia_medio = goertzel(muestras, FREQ_MEDIO, FS)
    energia_agudo = goertzel(muestras, FREQ_AGUDO, FS)

    filtro = decidir_filtro(
        energia_grave,
        energia_medio,
        energia_agudo
    )

    seleccionar_filtro(filtro)

    print("G:", int(energia_grave),
          "M:", int(energia_medio),
          "A:", int(energia_agudo),
          "RELE:", filtro)
```